

Universidad de la Fuerzas Armadas - ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software



Análisis y Diseño de Software - NRC30723

Proyecto: Sistema Bocatto

Título: GUÍA 2 PARA DESARROLLAR EN PLANTUML:

“Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis”

Integrantes: Carlos Caiza, Kerlly Chiriboga, Alexander Villacres

Tutor: Mgtr. Jenny Ruiz

Fecha de entrega: 4 de Mayo, 2026

GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Modelado De Análisis Con Plantuml (Cu0-0 A Cu0-3)
Caso base	Gestión de requisitos funcionales según el ERS de Sistema BOCATTO.
Operaciones CRUD	Agregar, Actualizar, Eliminar y Mostrar Todo
Referencia base	Guia_PlantUML_CU_Analisis_Grupo2
Propósito	Realizar un análisis estructural y dinámico a partir del análisis funcional previamente realizado, manteniendo coherencia entre requisito, caso de uso, clase y secuencia.

1. Propósito de la guía

Esta guía tiene como finalidad orientar la transición desde el análisis funcional inicial hacia el análisis estructural y dinámico del Sistema Bocatto. El énfasis no radica en la implementación técnica o de interfaz, sino en la comprensión de las responsabilidades de cada objeto bajo el patrón BCE (Boundary-Control-Entity).

Se busca garantizar la trazabilidad integral, asegurando que cada interacción modelada en los diagramas de secuencia sea coherente con los requisitos funcionales de nivel 0 y con las operaciones definidas en el diagrama de clases de análisis.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en esta Guía
Casos de uso de análisis	¿Qué necesita hacer el actor con el sistema?	Diagrama de casos de uso y matriz RF-CU.	Permite identificar responsabilidades, clases frontera, control y entidad.
Clases de análisis	¿Qué objetos participan para cumplir los casos de uso?	Diagrama de clases de análisis con estereotipos boundary, control, entity y repositorio.	Define los objetos (Frontera, Control, Entidad, Repositorio) que aparecerán en las secuencias.
Secuencias de análisis	¿Cómo colaboran actor, frontera, control y entidad para ejecutar cada caso de uso?	Diagramas de secuencia por operación de pedido del cliente o personal	Valida que el flujo dinámico sea coherente con los mensajes declarados en las clases.

3. Caso base actualizado: Pedidos (Sistema Bocatto)

El caso se basa en interfaces para la toma de pedidos de los clientes, con varios campos de entrada y tres posibles tipos de pedido.

Dato	Tipo sugerido	Descripción	Validación mínima
Número de mesa	Texto corto o entero	Identificador único de la mesa..	Obligatorio
Productos seleccionados	Texto	Producto seleccionado para el pedido	Obligatorio; por lo menos un producto
Cantidad por producto	Entero	Cantidad seleccionada de cierto producto.	Obligatoria, numérica y mayor que 0.

4. Requisitos Funcionales De Nivel 0

Basados en la ERS de Bocatto, se identifican los siguientes requerimientos para el desarrollo de la Guía:

Código	Requisito Funcional	Actor Principal	Caso de Uso
RF-00	Acceso seguro mediante usuario y clave.	Usuario (General)	CU0-0 Autenticar en el sistema
RF-01	Registro de pedidos directamente en la mesa.	Mesero	CU0-1 Tomar pedido en mesa
RF-02	Toma de pedidos vía telefónica para delivery.	Operador	CU0-2 Tomar pedido telefónico
RF-03	Autogestión de pedido mediante código QR.	Cliente	CU0-3 Realizar pedido por QR

5. Diagrama De Casos De Uso De Análisis

Representación del límite del sistema y los actores involucrados en los procesos de pedido.

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Mesero" as M
actor "Operador" as O
actor "Cliente" as C
actor "Cocinero" as CO

rectangle "Sistema Bocatto - Gestión de Pedidos" {
usecase "CU0-0: Autenticar en el sistema" as UC0
usecase "CU0-1: Tomar pedido en mesa" as UC1
usecase "CU0-2: Tomar pedido telefónico" as UC2
usecase "CU0-3: Realizar pedido por QR" as UC3
usecase "Validar datos del pedido" as UC_Val
usecase "CU0-4: Gestionar preparación del pedido" as UC4

usecase "Revisar lista de pedidos pendientes" as UC4_Review
```

```

usecase "Seleccionar y actualizar estado del pedido" as UC4_Update
}

M --> UC1
O --> UC2
C --> UC3
CO -right-> UC4

UC1 ..> UC_Val : <<include>>
UC2 ..> UC_Val : <<include>>
UC3 ..> UC_Val : <<include>>

UC1 ..> UC0 : <<include>>
UC2 ..> UC0 : <<include>>
UC4 ..> UC0 : <<include>>

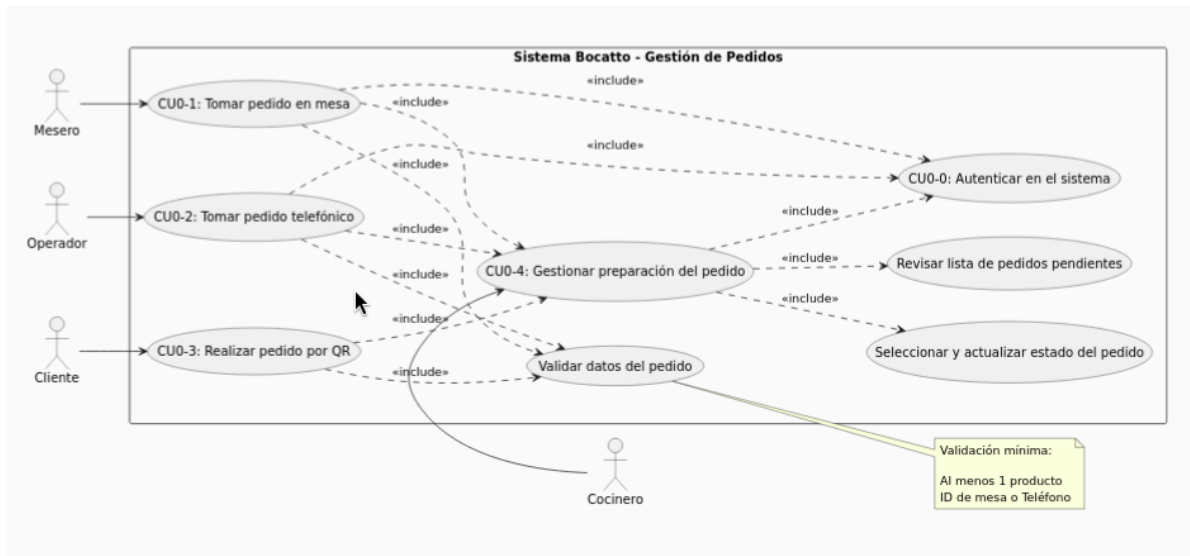
' CU0-1..CU0-3 include CU0-4 (preparation management is part of taking orders)
UC1 ..> UC4 : <<include>>
UC2 ..> UC4 : <<include>>
UC3 ..> UC4 : <<include>>

' CU0-4 composition
UC4 ..> UC4_Review : <<include>>
UC4 ..> UC4_Update : <<include>>

note right of UC_Val
Validación mínima:
Al menos 1 producto
ID de mesa o Teléfono
end note

@enduml

```



6. Derivación de Clases de Análisis

Tipo de clase	Responsabilidad en el análisis	Clases propuesta	Relación con los casos de uso
Boundary / Frontera	Recibe datos del actor y muestra resultados.	FormularioLogin	Interfaz para el usuario para acceder al sistema mediante usuario y clave.
		InterfazPedidoMesa	Interfaz para el mesero para registrar un pedido para una mesa.
		InterfazPedidoTel	Interfaz para el cliente para realizar un pedido por delivery con un número telefónico.
		AppMovilQR	Interfaz para el cliente para realizar un pedido usando un código QR localizado en la mesa.
Control	Coordina la lógica de cada caso de uso.	GestorAutenticacion	Valida que las credenciales ingresadas sean válidas.
		ControlRegistroPedidos	Administra los pedidos registrados por los usuarios.
Entity / Entidad	Representa la información del dominio.	Usuario	Superentidad que incluye a los clientes, meseros y operadores.
		Pedido	Conjunto de platos solicitados por los clientes.
		Plato	Entidad que representa uno de los diferentes platos disponibles.
		Cliente	Entidad que realiza pedidos.
Repositorio / Colección	Conserva y recupera registros.	RepositorioUsuarios	Permite validar las credenciales de los usuarios para brindar acceso a cada uno a su respectiva interfaz.

```

@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

' --- FRONTERAS (Boundary) ---
class "FormularioLogin" <<boundary>> {
+ ingresarCredenciales(user, pass)
+ mostrarMenuPrincipal()
}

class "InterfazPedidoMesa" <<boundary>> {
- txtMesa: String
+ capturarMesaYProductos(mesa, lista)
+ mostrarConfirmacion()
}

class "InterfazPedidoTel" <<boundary>> {
- txtTelefono: String
+ capturarDatosCliente(tel, dir)
+ cargarProductos(lista)
+ mostrarExitoDelivery()
}

class "AppMovilQR" <<boundary>> {

```

```

+ leerCodigoQR(mesaID)
+ gestionarCarrito(items)
+ presentarMenuDigital()
}

' --- CONTROLES (Control) ---
class "GestorAutenticacion" <<control>> {
+ autenticarEnElSistema(user, pass): boolean
+ validarCredenciales(user, pass): boolean
+ identificarRol(user): Rol
}

class "ControlRegistroPedido" <<control>> {
+ tomarPedidoEnMesa(mesa, lista)
+ tomarPedidoTelefonico(tel, lista)
+ realizarPedidoPorQR(mesaID, lista)
+ validarDatosDelPedido(lista): boolean
}

' --- ENTIDADES (Entity) ---
' Se cambia Plato por Menu según instrucción [1]
class "Menu" <<entity>> {
- idProducto: String
- nombre: String
- precio: double
+ consultarItemsDisponibles(): List<Menu>
}

class "Usuario" <<entity>> {
- user: String
- pass: String
- rol: String
+ compararClave(pass): boolean
}

class "Pedido" <<entity>> {
- id: String
- mesa: String
- estado: String
+ crearNuevoPedido(mesa)
}

class "Cliente" <<entity>> {
- telefono: String
- direccion: String
+ validarClienteExistente(tel): Cliente
}

' --- REPOSITARIOS (Repository) ---
class "RepositorioUsuarios" <<repository>> {
+ buscarUsuarioPorNombre(user): Usuario
}

class "RepositorioPedidos" <<repository>> {
+ guardarPedido(pedido: Pedido): void
+ buscarPedidoPorId(id: String): Pedido
}

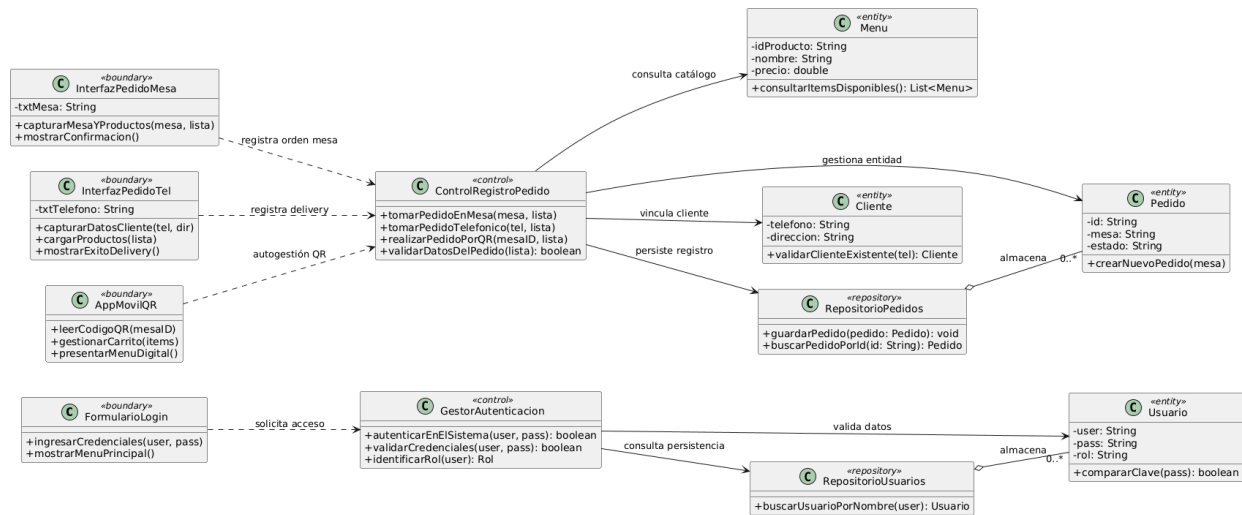
' --- RELACIONES DE TRAZABILIDAD ---
FormularioLogin ..> GestorAutenticacion : solicita acceso
InterfazPedidoMesa ..> ControlRegistroPedido : registra orden mesa
InterfazPedidoTel ..> ControlRegistroPedido : registra delivery
AppMovilQR ..> ControlRegistroPedido : autogestión QR

GestorAutenticacion --> Usuario : valida datos
GestorAutenticacion --> RepositorioUsuarios : consulta persistencia

```

ControlRegistroPedido --> Pedido : gestiona entidad
 ControlRegistroPedido --> Menu : consulta catálogo
 ControlRegistroPedido --> Cliente : vincula cliente
 ControlRegistroPedido --> RepositorioPedidos : persiste registro

RepositorioUsuarios o-- "0..*" Usuario : almacena
 RepositorioPedidos o-- "0..*" Pedido : almacena
 @enduml



7. Matriz De Relación: Casos De Uso, Clases Y Secuencias

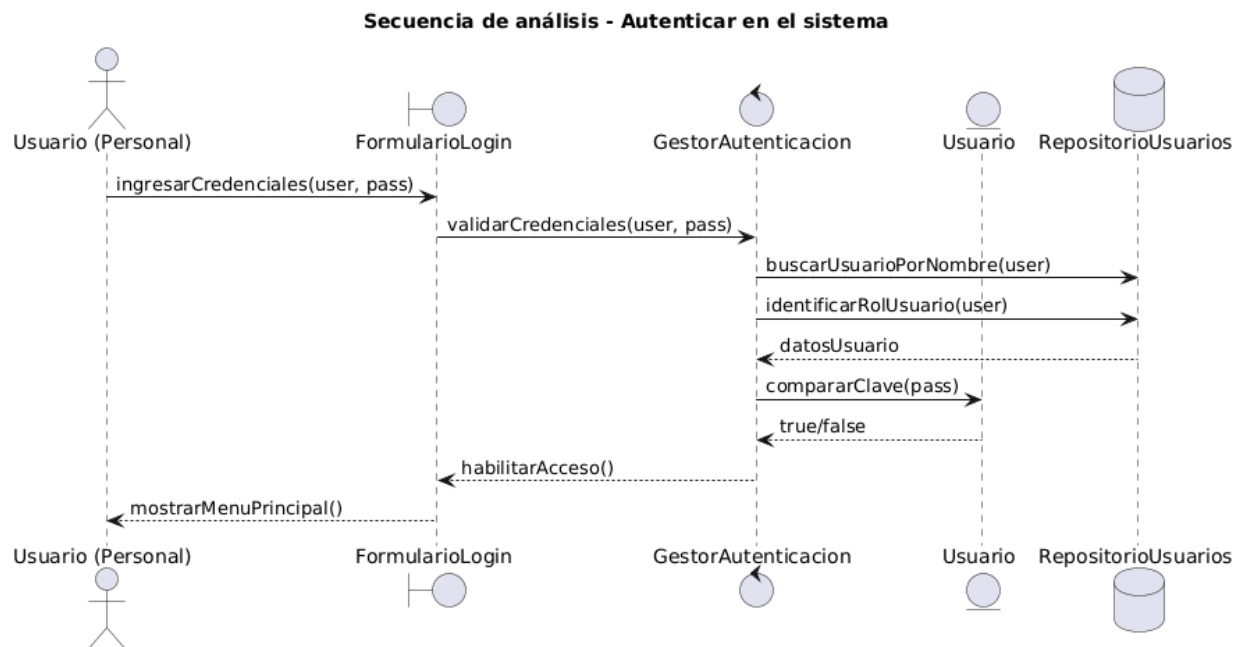
Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
CU0-0: Autenticar en el Sistema	FormularioLogin: Captura credenciales.	GestorAutenticacion : Ejecuta validarCredenciales y asigna el rol	Usuario / RepositorioUsuarios: Verifica credenciales en la colección de usuarios.	Ingresar datos → buscar usuario → comparar clave → identificar rol → habilitar acceso.
CU0-1: Tomar pedido en mesa	InterfazPedidoMesa: Captura mesa, menú y observaciones.	ControlRegistroPedido: Ejecuta tomarPedidoEnMesa y valida datos	Pedido, Menu, Mesero / RepositorioPedidos: Crea y persiste la orden	Autenticación previa → Ingresar mesa → Seleccionar productos → Validar (mín. 1 producto del menú) → Registrar → Guardar.
CU0-2: Tomar pedido telefónico	InterfazPedidoTel: Captura datos de contacto, dirección y orden.	ControlRegistroPedido: Ejecuta tomarPedidoTelefonico	Cliente, Pedido, Menu / RepositorioPedidos: Vincula al cliente con su orden.	Autenticación previa → Validar cliente → Seleccionar → Validar datos → Registrar pedido → Guardar.
CU0-3: Realizar pedido por QR	AppMovilQR: Identifica mesa mediante QR y captura productos.	ControlRegistroPedido: Ejecuta realizarPedidoPorQR	Pedido, Menu, Cliente / RepositorioPedidos: Entidades para la autogestión.	Escanear QR → Mostrar menú → Seleccionar productos → Validar (mín. 1 producto del menú) → Confirmar → Guardar.

8. Diagramas De Secuencia De Análisis

8.1 CU0-0: Autenticar en el sistema

```
@startuml
title Secuencia de análisis - Autenticar en el sistema
actor "Usuario (Personal)" as Actor
boundary "FormularioLogin" as UI
control "GestorAutenticacion" as Ctrl
entity "Usuario" as Ent
database "RepositorioUsuarios" as Repo

Actor -> UI : ingresarCredenciales(user, pass)
UI -> Ctrl : validarCredenciales(user, pass)
Ctrl -> Repo : buscarUsuarioPorNombre(user)
Ctrl -> Repo : identificarRolUsuario(user)
Repo --> Ctrl : datosUsuario
Ctrl -> Ent : compararClave(pass)
Ent --> Ctrl : true/false
Ctrl --> UI : habilitarAcceso()
UI --> Actor : mostrarMenuPrincipal()
@enduml
```



8.2 CU0-1: Tomar pedido en mesa

```
@startuml
title Secuencia de análisis - Tomar pedido en mesa
actor Mesero as Actor
boundary InterfazPedidoMesa as UI
```

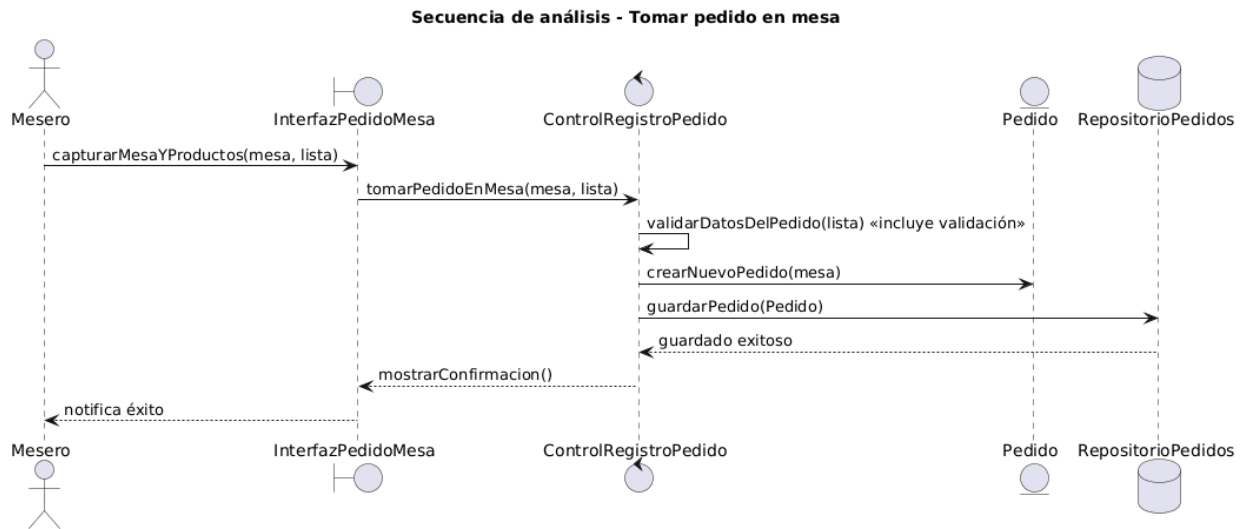


```

control ControlRegistroPedido as Ctrl
entity Pedido as Ent
database RepositorioPedidos as Repo

Actor -> UI : capturarMesaYProductos(mesa, lista)
UI -> Ctrl : tomarPedidoEnMesa(mesa, lista)
Ctrl -> Ctrl : validarDatosDelPedido(lista) <<incluye validación>>
Ctrl -> Ent : crearNuevoPedido(mesa)
Ctrl -> Repo : guardarPedido(Pedido)
Repo --> Ctrl : guardado exitoso
Ctrl --> UI : mostrarConfirmacion()
UI --> Actor : notifica éxito
@enduml

```



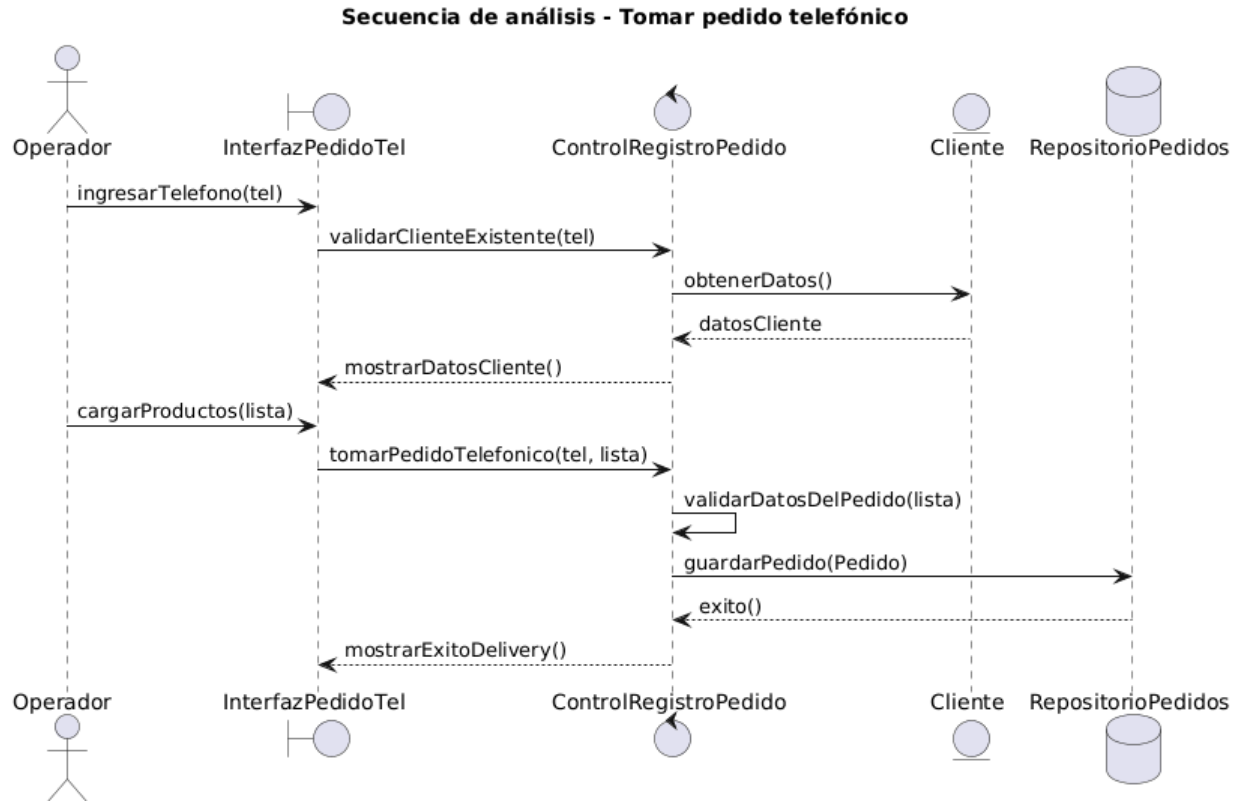
8.3 CU0-2: Tomar pedido telefónico

```

@startuml
title Secuencia de análisis - Tomar pedido telefónico
actor Operador as Actor
boundary InterfazPedidoTel as UI
control ControlRegistroPedido as Ctrl
entity Cliente as Ent
database RepositorioPedidos as Repo

Actor -> UI : ingresarTelefono(tel)
UI -> Ctrl : validarClienteExistente(tel)
Ctrl -> Ent : obtenerDatos()
Ent --> Ctrl : datosCliente
Ctrl --> UI : mostrarDatosCliente()
Actor -> UI : cargarProductos(lista)
UI -> Ctrl : tomarPedidoTelefonico(tel, lista)
Ctrl -> Ctrl : validarDatosDelPedido(lista)
Ctrl -> Repo : guardarPedido(Pedido)
Repo --> Ctrl : exito()
Ctrl --> UI : mostrarExitoDelivery()
@enduml

```



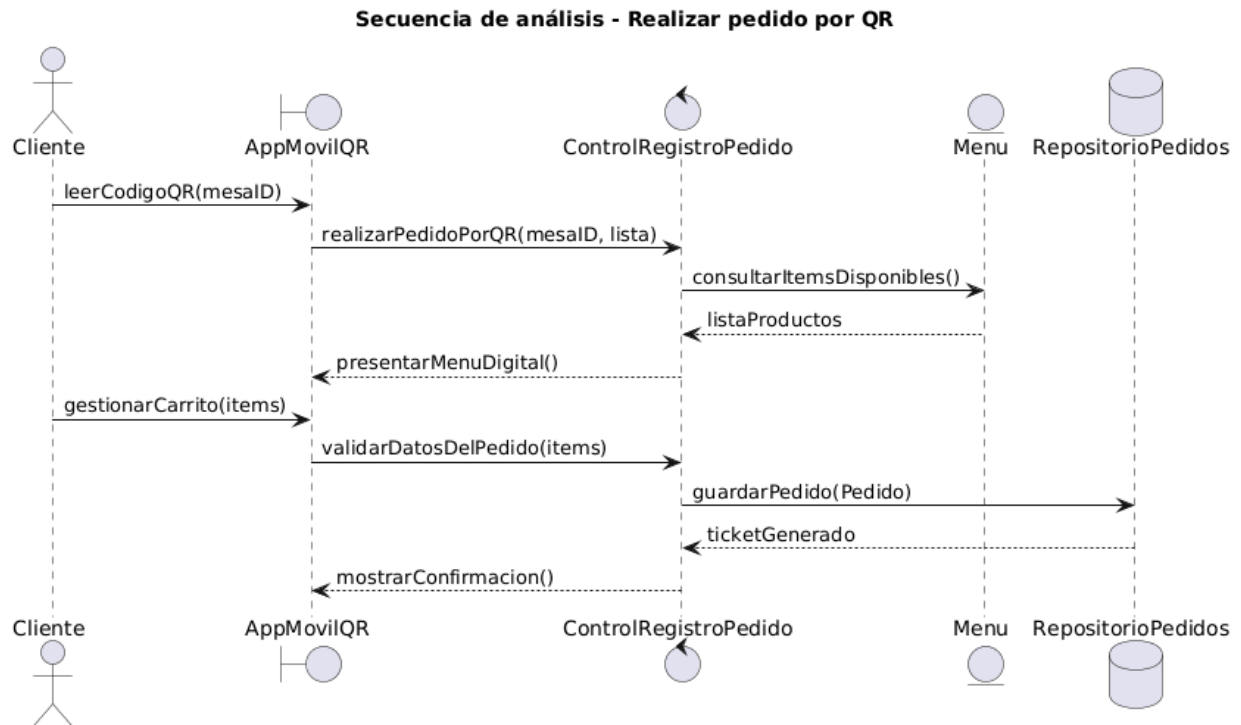
8.4 CU0-3: Realizar pedido por QR

```

@startuml
title Secuencia de análisis - Realizar pedido por QR
actor Cliente as Actor
boundary AppMovilQR as UI
control ControlRegistroPedido as Ctrl
entity Menu as Ent
database RepositorioPedidos as Repo

Actor -> UI : leerCodigoQR(mesaID)
UI -> Ctrl : realizarPedidoPorQR(mesaID, lista)
Ctrl -> Ent : consultarItemsDisponibles()
Ent --> Ctrl : listaProductos
Ctrl --> UI : presentarMenuDigital()
Actor -> UI : gestionarCarrito(items)
UI -> Ctrl : validarDatosDelPedido(items)
Ctrl -> Repo : guardarPedido(Pedido)
Repo --> Ctrl : ticketGenerado
Ctrl --> UI : mostrarConfirmacion()
@enduml

```



9. Matriz De Trazabilidad (Rf - Cu - Clases)

RF	Caso de Uso	Objetos (BCE + Repositorio)	Prioridad
RF-00	CU0-0: Autenticar en el sistema	FormularioLogin, GestorAutenticacion, Usuario, RepositorioUsuarios	Alta
RF-01	CU0-1: Tomar pedido en mesa	InterfazPedidoMesa, ControlRegistroPedido, Pedido, Menu, RepositorioPedidos	Alta
RF-02	CU0-2: Tomar pedido telefónico	InterfazPedidoTel, ControlRegistroPedido, Cliente, Pedido, Menu, RepositorioPedidos	Media
RF-03	CU0-3: Realizar pedido por QR	AppMovilQR, ControlRegistroPedido, Pedido, Menu, Cliente, RepositorioPedidos	Alta

10. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
CU0-0: Autenticar en el sistema	El usuario debe estar registrado previamente con un rol asignado.	Ingresar credenciales -> validar acceso -> identificar rol -> habilitar funcionalidades.	Acceso concedido y menú mostrado según el rol del usuario.
CU0-1: Tomar pedido en mesa	El mesero debe haber iniciado sesión correctamente en el sistema.	Seleccionar mesa y platos -> validar datos (mín. 1 producto) -> registrar orden -> generar ID -> enviar a cocina.	Pedido registrado con estado "pendiente" y enviado a cocina para preparación.
CU0-2: Tomar pedido telefónico	El operador debe estar autenticado en su estación de trabajo.	Ingresar teléfono del cliente -> validar cliente -> seleccionar productos -> validar datos -> registrar pedido.	Pedido para delivery registrado correctamente con estado inicial "pendiente".
CU0-3: Realizar pedido por QR	El cliente debe contar con un dispositivo móvil con cámara y conexión a internet.	Escanear código QR -> ver menú digital -> seleccionar productos -> validar datos -> confirmar carrito -> enviar orden.	Pedido autogestionado correctamente, asociado a la mesa y visible en cocina.

11. Conclusión Técnica

El modelo de análisis desarrollado para el Sistema Bocatto garantiza una coherencia técnica integral, ya que asegura que los requisitos funcionales (“qué”) se transformen en responsabilidades estructurales mediante el patrón BCE (“quiénes”) y se validen a través de diagramas de secuencia (“cómo”). La principal utilidad de este enfoque radica en la trazabilidad estricta: cada mensaje modelado en las secuencias dinámicas (como validarDatos o guardar) corresponde exactamente a una operación declarada en las clases de análisis y en la capa de persistencia de los Repositorios, eliminando ambigüedades antes de la implementación.

Esta versión del modelo refuerza la robustez del sistema al integrar la lógica de validación obligatoria (mínimo un producto) como un comportamiento incluido (<<include>>) y al establecer una dependencia de autenticación para el personal interno, asegurando que las funciones críticas no operen como piezas aisladas. La inclusión de Repositorios cierra el ciclo de análisis al definir cómo se recupera y conserva la información del dominio.

En conclusión, el uso de PlantUML como herramienta de “Diagram as Code” permitió verificar que el sistema no solo es funcionalmente completo, sino estructuralmente sólido y modular. Esto facilita su futura escalabilidad y mantenimiento al centralizar la lógica de negocio en controladores compartidos y entidades bien definidas, cumpliendo con los estándares de calidad exigidos en la ruta de análisis.